

July 05, 2026

Checks-in, and Balances: a Matter of Time

Read Me

Yash Moitra[†]

[†] Delhi International Airport Limited.

Contents

I.	The question, in one paragraph	1
II.	Headline findings	1
III.	How the model is built (five layers)	1
IV.	Reading order for a new reader	2
V.	Rebuilding everything	2
VI.	Data provenance and integrity	3
VII.	Repository packaging note	3
VIII.	Authorship	3

A passenger check-in forecast and counter-allocation model for international departures at Delhi Indira Gandhi International Airport (DEL), Terminal 3, built on the schedule of a single operating day: Day 7, June 21, 2026. The model answers a practical question: given when passengers show up, how many check-in counters does Terminal 3 need, minute by minute, to keep every passenger's wait below target?

An interactive companion dashboard lives at `dashboard/index.html` (also served online; see the repository page). All formatted documents are Computer Modern PDFs rendered through LaTeX, per the project ground rules.

I The question, in one paragraph

On June 21, 2026, Terminal 3 dispatches 164 international flights carrying at most 39,610 seats, spread across 57 airlines, 65 destinations, and 4 ground handlers. Every passenger must pass a check-in counter inside a fixed window: counters for a flight open four hours before departure and close one hour before. When passengers arrive inside that window is not observed, so the model treats arrival behaviour as the experimental variable: 104 candidate entry-time distributions (Normals, a Uniform, exponential growth and decay shapes, and left and right skew-normals) are instantiated for every seat. A verified FIFO wait engine then computes, for each behaviour, each service speed, and each way of pooling counters, the minute-by-minute counter requirement, which is finally compared against the terminal's physical supply of 168 counters across 13 islands.

II Headline findings

1. At a service speed of 180 seconds per passenger, the OMDA 20-minute economy wait target fits inside the 168-counter supply under every pooling arrangement, for the reference arrival behaviour N1 (arrivals centred 150 minutes before departure, SD 30).
2. At 200 seconds per passenger with counters pooled per flight, the same target needs 171 counters, 3 more than exist. Pooling by airline (164) or by ground handler (159) absorbs the peak.
3. Arrival behaviour, not counter count, governs feasibility. Across all 104 arrival shapes the per-flight peak requirement ranges from 136 to 229 counters (r180, 20-minute target). Back-loaded, last-minute arrival shapes breach any realistic supply.
4. The demand day is double-peaked: seat capacity peaks after midnight (2,507 seats in the 00:00 hour) and again at midday (2,404 in the 11:00 hour, with the most flights, 11, in the 12:00 hour); the quietest hour is 17:00 (350 seats).
5. Ground handling is concentrated: AISATS handles 83 flights (20,203 seats) and BFS 71 (16,602); Ramp 360 and AIESL carry the remainder.

Full derivations are in `Methodology.md` and, for the wait and allocation layer, `assumed statics/Methodology_Counter_Allocation.md`. Computed exhibits are in `INSIGHTS.md`.

III How the model is built (five layers)

Layer	Content	Status
A. Schedule foundation	asimplified.csv : 164 flights, one row per flight, cleaned (flight B4 443 omitted, provenance preserved)	Built and verified
B. Seat universe	seats.csv : one row per occupiable seat, $N = 39,610$, many-to-one onto flights	Built and verified
C. Entry times	104 models over the check-in window, one column per model in seats.xlsx , all moment-verified	Built and verified
D. Desks and service	FIFO wait engine, two service regimes (180 s and 200 s per passenger)	Built and verified
E. Allocation vs supply	Time-varying counter rule, three poolings, two targets (20 and 40 min), compared to 168 counters	Built and verified

Still open, recorded in the methodology documents: load factors below 1, desk eligibility by cabin class and loyalty tier, island-level supply constraints (14 per island), and the tighter time-varying optimum.

IV Reading order for a new reader

1. README.md (this file), then Model_Overview.md for the same story with equations.
2. 00_Ground_Rules.md: the ten conventions every document follows.
3. Methodology.md: Steps 1 and 2 (seat universe and entry times), Equations (1) to (6).
4. assumed statics/Methodology_Counter_Allocation.md: Steps 3 and 4 (waits and allocation), Equations (7) to (16), results and limitations.
5. Variable_Names.md and Definitions.md: the living registries.
6. DATA_DICTIONARY.md: every column of every data file.
7. dashboard/index.html: the interactive view of all of the above.

FOLDER_INDEX.md maps every file. Formatted PDF renders of the governance documents are in docs/.

V Rebuilding everything

Artefact	Command
Seat universe (seats.csv)	<code>python3 build_passengers.py</code>
Governance PDFs (docs/)	<code>bash build_docs.sh</code> (pandoc + XeLaTeX, Latin Modern fonts)
Wait engine self-test	<code>python3 "assumed statics/wait_time.py"</code>

Artefact	Command
Schedule and curve renders	Re-knit the corresponding <code>.Rmd</code> (R, knitr, pdfflatex/XeLaTeX)

The per-model renders in `0623.2203 renderings/` (ten files per model, 104 models) were generated from the same `.Rmd` pattern; any one of them can be re-knit independently.

VI Data provenance and integrity

The schedule was received as `asrecd.csv`, cleaned into `asimplified.csv` (the version every computation uses), with the pre-removal original preserved as `asimplified_with_B4_443_raw.csv`. The seat universe is deterministic given the schedule. Entry-time draws are seeded (seeds recorded per model in `model_equations.md` and in the `reference` tab of `seats.xlsx`); `verification_report.` certifies all 104 models against their target moments (104 of 104 PASS). Original worksheet photographs are kept in `original_photos/`.

VII Repository packaging note

In the Git repository the 19 MB seat-level master `seats.xlsx` travels as exact binary parts (`seats.xlsx.part-00.bin` to `-03.bin`) because of a transfer-size limit in the publication pipeline. Run `python3 rejoin_seats.py` once after cloning (or concatenate the parts in order) to reconstruct `seats.xlsx`; the script verifies the SHA-256. In the working folder the file is present whole. The redundant `pre0618/Photos-3-001.zip` is not carried in the repository; see `pre0618/README.md`.

VIII Authorship

All modelling, documentation, data preparation, and code in this repository are the work of **Yash Moitra** (Cornell University), produced for Delhi International Airport Limited. Every PDF page carries a watermark to that effect. Please retain attribution when reusing or extending any part of this work.